

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра компьютерных и информационных наук

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7

дисциплина: Архитектура компьютеров

Студент: Симакова Алина

Группа: НКАбд-05-25

МОСКВА

2025 г.

Оглавление

1 Цель работы.....	3
2 Задание.....	4
3 Теоретическое введение.....	5
4 Выполнение лабораторной работы.....	6
4.1 Реализация переходов в NASM.....	6
4.2 Изучение структуры файла листинга.....	11
5 Задания для самостоятельной работы.....	14
6 Выводы.....	20
Список литературы.....	21

1 Цель работы

Целью данной лабораторной работы является изучение команд и безусловных переходов, а также приобретение навыков написания программ с использованием переходов и знакомство с назначением, структурой файла листинга.

2 Задание

На основе методических указаний освоить реализацию переходов в NASM, изучить структуры файлов листинга и выполнить самостоятельное написание программ по материалам лабораторной работы.

3 Теоретическое введение

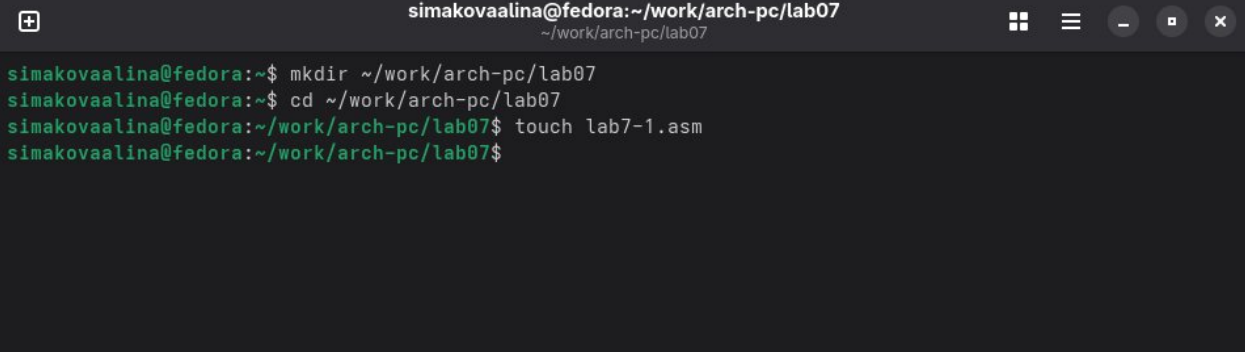
Для реализации ветвлений в ассемблере используются так называемые команды передачи управления или команды перехода. Можно выделить 2 типа переходов:

- *условный переход* – выполнение или не выполнение перехода в определенную точку программы в зависимости от проверки условия;
- *безусловный переход* – выполнение передачи управления в определенную точку программы без каких-либо условий.

4 Выполнение лабораторной работы

4.1 Реализация переходов в NASM

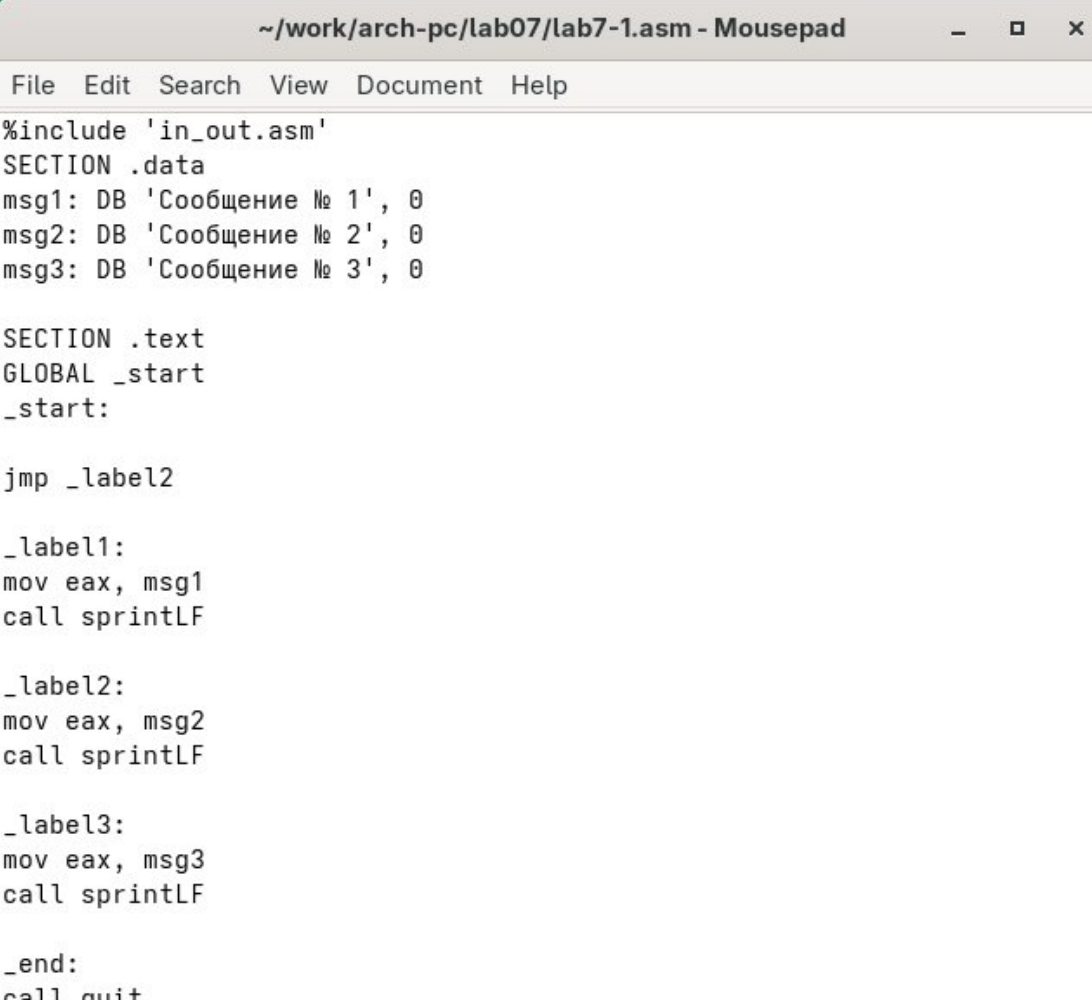
В домашней директории создаю каталог для программ лабораторной работы №7 (рис. 4.1).



```
simakovaalina@fedora:~/work/arch-pc/lab07
simakovaalina@fedora:~$ mkdir ~/work/arch-pc/lab07
simakovaalina@fedora:~$ cd ~/work/arch-pc/lab07
simakovaalina@fedora:~/work/arch-pc/lab07$ touch lab7-1.asm
simakovaalina@fedora:~/work/arch-pc/lab07$
```

Рис. 4.1: Создание каталога и файла для программы

Далее копирую код из листинга в файл будущей программы (рис. 4.2).



```
~/work/arch-pc/lab07/lab7-1.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'
SECTION .data
msg1: DB 'Сообщение № 1', 0
msg2: DB 'Сообщение № 2', 0
msg3: DB 'Сообщение № 3', 0

SECTION .text
GLOBAL _start
_start:

jmp _label2

_label1:
mov eax, msg1
call sprintLF

_label2:
mov eax, msg2
call sprintLF

_label3:
mov eax, msg3
call sprintLF

_end:
call quit
```

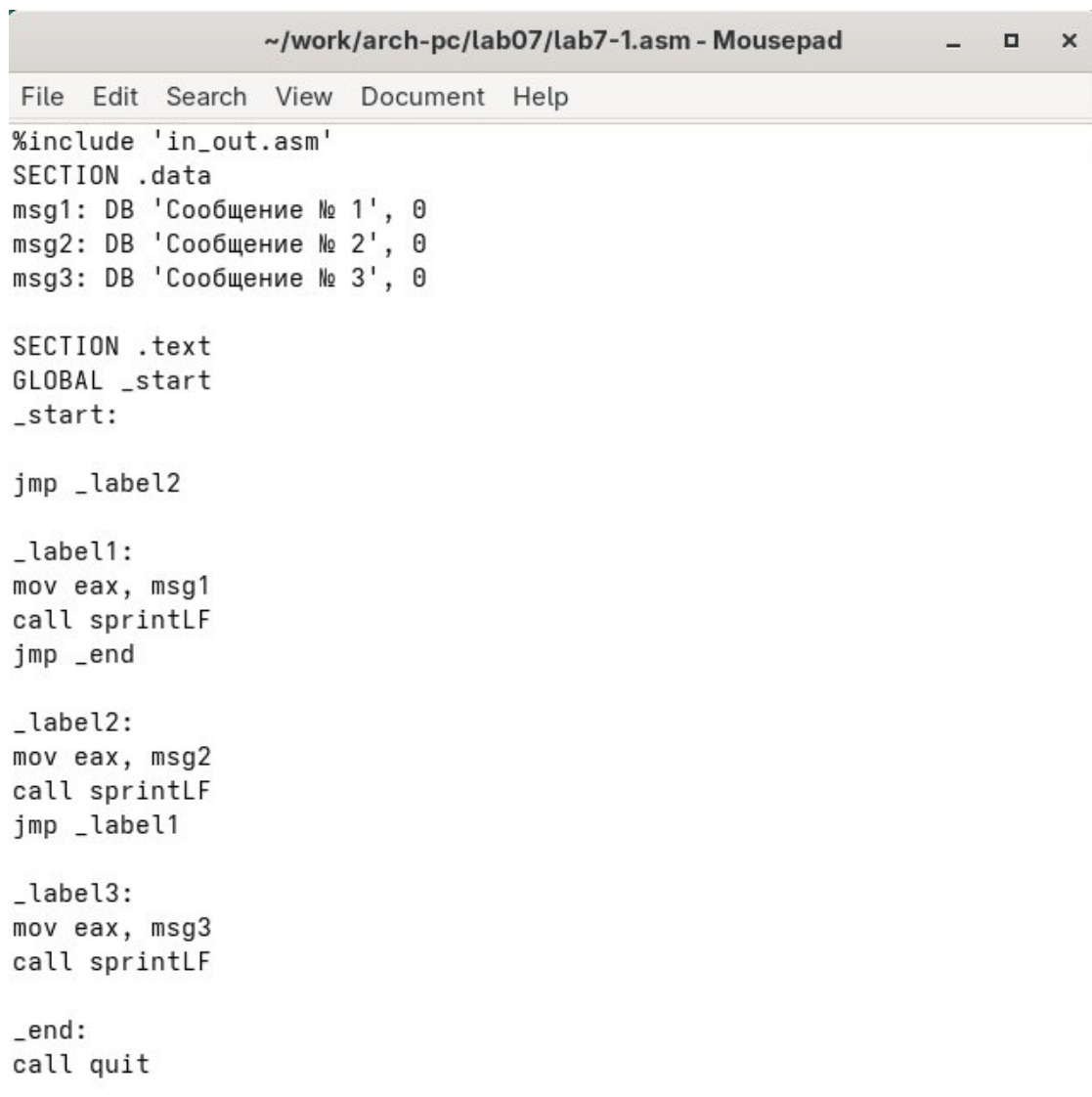
Рис. 4.2: Сохранение программы

При запуске программы я убедилась в том, что не условный переход действительно изменяет порядок выполнения инструкций (рис. 4.3).

```
simakovaalina@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
simakovaalina@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
simakovaalina@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 3
simakovaalina@fedora:~/work/arch-pc/lab07$
```

Рис. 4.3: Запуск программы

Изменяю программу таким образом, чтобы поменялся порядок выполнения функций (рис. 4.4).



```
~/work/arch-pc/lab07/lab7-1.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'
SECTION .data
msg1: DB 'Сообщение № 1', 0
msg2: DB 'Сообщение № 2', 0
msg3: DB 'Сообщение № 3', 0

SECTION .text
GLOBAL _start
_start:

jmp _label2

_label1:
mov eax, msg1
call sprintf
jmp _end

_label2:
mov eax, msg2
call sprintf
jmp _label1

_label3:
mov eax, msg3
call sprintf

_end:
call quit
```

Рис. 4.4: Изменение программы

Теперь запускаю программу и проверяю, что примененные изменения верны (рис. 4.5).

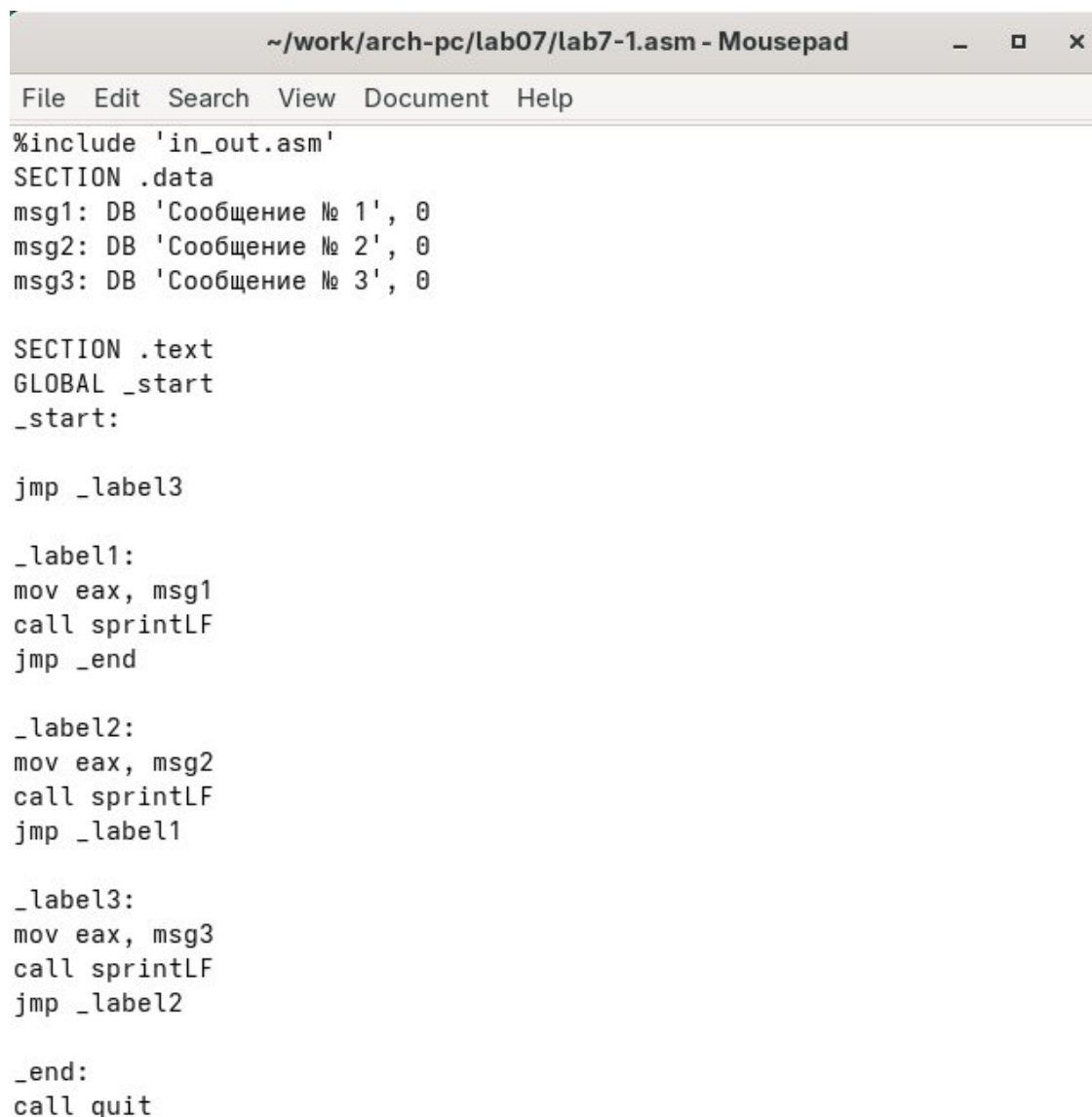
```

simakovaalina@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
simakovaalina@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
simakovaalina@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 1
simakovaalina@fedora:~/work/arch-pc/lab07$

```

Рис. 4.5: Запуск измененной программы

Изменяю текст программы так, чтобы все три сообщения вывелись в обратном порядке (рис. 4.6).



```

~/.work/arch-pc/lab07/lab7-1.asm - Mousepad
File Edit Search View Document Help
%include 'in_out.asm'
SECTION .data
msg1: DB 'Сообщение № 1', 0
msg2: DB 'Сообщение № 2', 0
msg3: DB 'Сообщение № 3', 0

SECTION .text
GLOBAL _start
_start:

jmp _label3

_label1:
mov eax, msg1
call sprintf
jmp _end

_label2:
mov eax, msg2
call sprintf
jmp _label1

_label3:
mov eax, msg3
call sprintf
jmp _label2

_end:
call quit

```

Рис. 4.6: Изменение программы

Работа выполнена корректно, и программа в нужном мне порядке выводит сообщения (рис. 4.7).


```

simakovaalina@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
simakovaalina@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
simakovaalina@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1
simakovaalina@fedora:~/work/arch-pc/lab07$

```

Рис. 4.7: Проверка изменений

Создаю новый рабочий файл и вставляю в него код из следующего листинга (рис. 4.8).

```

~/work/arch-pc/lab07/lab7-2.asm - Mousepad
File Edit Search View Document Help
#include "in_out.asm"

SECTION .data
msg1 db 'Введите B: ', 0
msg2 db 'Наибольшее число: ', 0
A dd '20'
C dd '50'

SECTION .bss
max resb 10
B resb 10

SECTION .text
GLOBAL _start
_start:

mov eax, msg1
call sprint

mov ecx, B
mov edx, 10
call sread

mov eax, B
call atoi
mov [B], eax

mov ecx, [A]
mov [max], ecx

cmp ecx, [C]
jg check_B
mov ecx, [C]
mov [max], ecx

check_B:
mov eax, max
call atoi
mov [max], eax

```

```

mov ecx, [max]
cmp ecx, [B]
jg fin
mov ecx, [B]
mov [max], ecx

fin:
mov eax, msg2
call sprint
mov eax, [max]
call iprintLF
call quit

```

Рис. 4.8: Сохранение новой программы

Программа выводит значение переменной с максимальным значением, проверяю работу программы с разным входными данными (рис. 4.9).

```

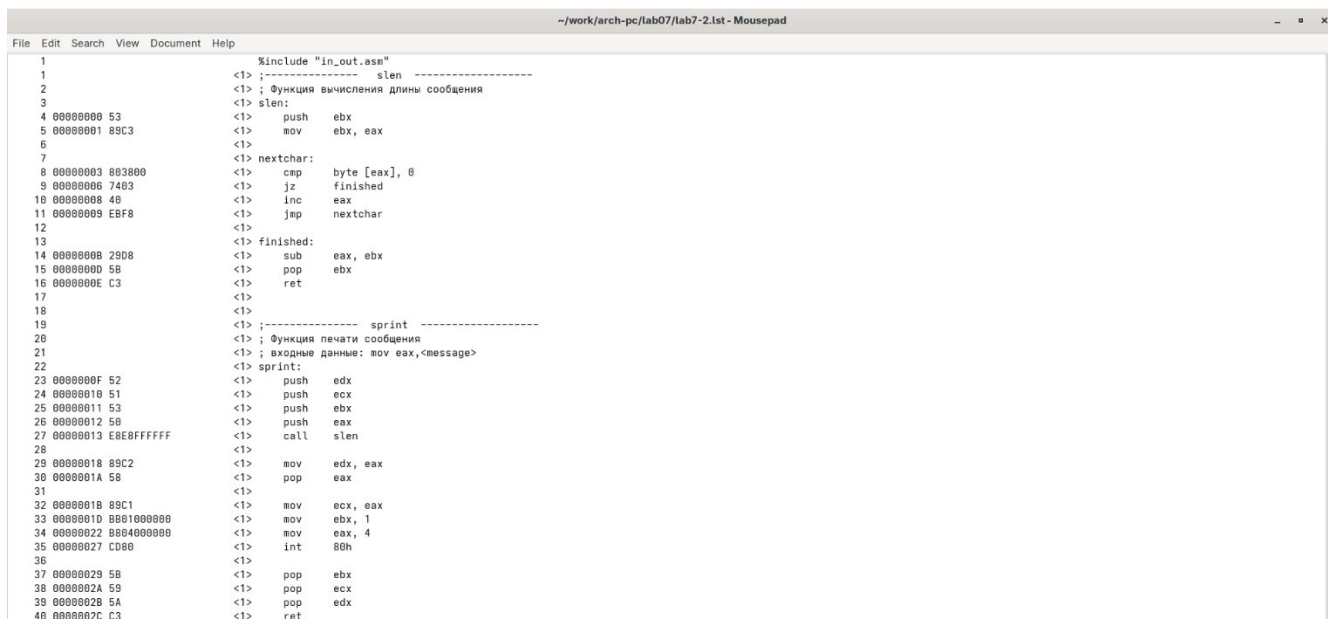
simakovaalina@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
simakovaalina@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
simakovaalina@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 20
Наибольшее число: 50
simakovaalina@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 60
Наибольшее число: 60
simakovaalina@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 15
Наибольшее число: 50
simakovaalina@fedora:~/work/arch-pc/lab07$

```

Рис. 4.9: Проверка программы из листинга

4.2 Изучение структуры файла листинга

Создаю файл листинга с помощью флага -l команды nasm и открываю его с помощью текстового редактора mousepad (рис. 4.10).

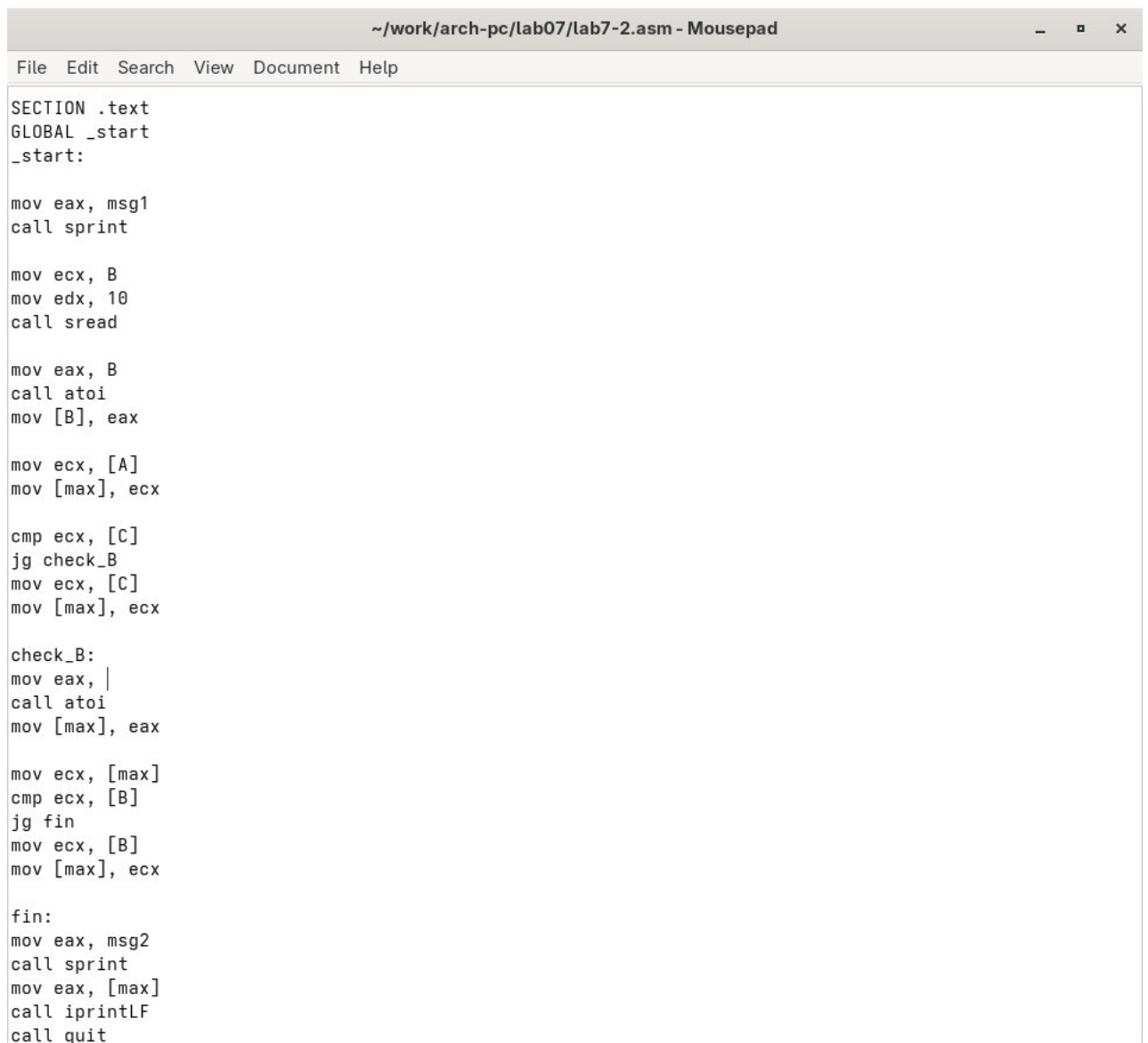


```
1                                %include "in_out.asm"
2                                <!------- slen ----->
3                                <!-- ; Функция вычисления длины сообщения
4                                <!-- slen:
5                                00000000 53          <!--      push     ebx
6                                00000001 89C3        <!--      mov      ebx, eax
7                                <!--
8                                <!-- nextchar:
9                                00000003 803800      <!--      cmp      byte [eax], 0
10                               00000006 7403        <!--      jz       finished
11                               00000008 40          <!--      inc      eax
12                               00000009 EBF8        <!--      jmp      nextchar
13                               <!--
14                               0000000B 29D8        <!--      finished:
15                               0000000D 5B          <!--      sub      eax, ebx
16                               0000000E C3          <!--      pop      ebx
17                               <!--
18                               <!--
19                               <!------- sprint ----->
20                               <!-- ; Функция печати сообщения
21                               <!-- ; входные данные: mov eax, <message>
22                               <!-- sprint:
23                               0000000F 52          <!--      push     edx
24                               00000010 51          <!--      push     ecx
25                               00000011 53          <!--      push     ebx
26                               00000012 58          <!--      push     eax
27                               00000013 E8E8FFFFFF    <!--      call     slen
28                               <!--
29                               00000018 89C2        <!--      mov      edx, eax
30                               0000001A 58          <!--      pop      eax
31                               <!--
32                               0000001B 89C1        <!--      mov      ecx, eax
33                               0000001D 0001000000    <!--      mov      ebx, 1
34                               00000022 0004000000    <!--      mov      eax, 4
35                               00000027 CD80        <!--      int      80h
36                               <!--
37                               00000029 5B          <!--      pop      ebx
38                               0000002A 59          <!--      pop      ecx
39                               0000002B 5A          <!--      pop      edx
40                               0000002C C3          <!--      ret
```

Рис. 4.10: Проверка файла листинга

Первое значение в файле листинга – номер строки, и он может вовсе не совпадать с номером строки изначального файла. Второе вхождение – адрес, смещение машинного кода относительно начала текущего сегмента, затем непосредственно идет сам машинный код, а заключает строку исходный текст программы с комментариями.

Удаляю один операнд из случайной инструкции, чтобы проверить поведение файла листинга в дальнейшем (рис. 4.11).



```
~/.work/arch-pc/lab07/lab7-2.asm - Mousepad
File Edit Search View Document Help

SECTION .text
GLOBAL _start
_start:

mov eax, msg1
call sprint

mov ecx, B
mov edx, 10
call sread

mov eax, B
call atoi
mov [B], eax

mov ecx, [A]
mov [max], ecx

cmp ecx, [C]
jg check_B
mov ecx, [C]
mov [max], ecx

check_B:
mov eax, |
call atoi
mov [max], eax

mov ecx, [max]
cmp ecx, [B]
jg fin
mov ecx, [B]
mov [max], ecx

fin:
mov eax, msg2
call sprint
mov eax, [max]
call iprintLF
call quit
```

Рис. 4.11: Удаление операнда из программы

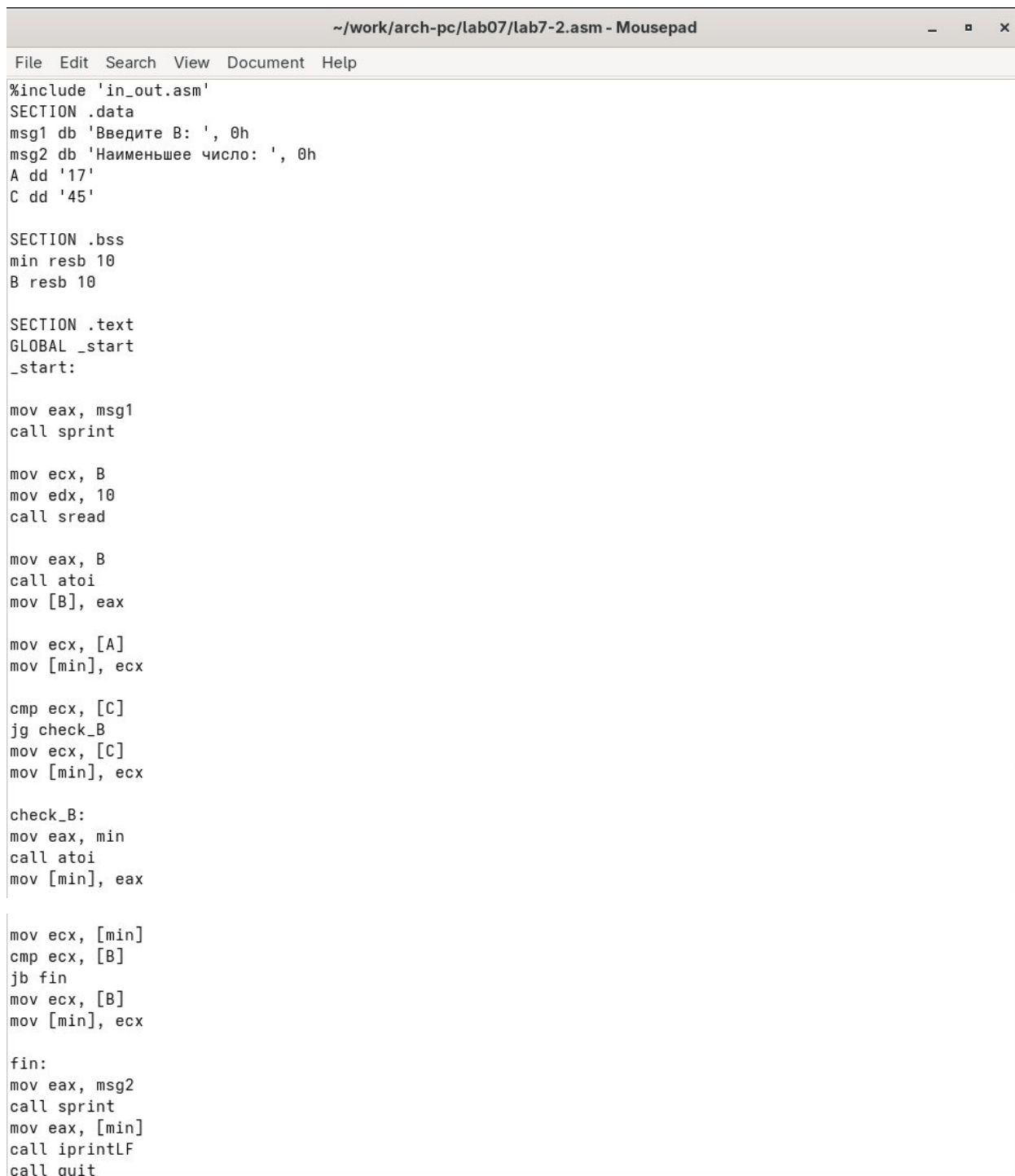
В новом файле листинга показывает ошибку, которая возникла при попытке трансляции файла. Никакие выходные файлы при этом помимо файла листинга не создаются (рис. 4.12).

```
~/work/arch-pc/lab07/lab7-2.lst - Mousepad
File Edit Search View Document Help
15
16 _start:
17 000000E8 B8[00000000] mov eax, msg1
18 000000ED E81DFFFFFF call sprint
19
20 000000F2 B9[0A000000] mov ecx, B
21 000000F7 BA0A000000 mov edx, 10
22 000000FC E842FFFFFF call sread
23
24 00000101 B8[0A000000] mov eax, B
25 00000106 E891FFFFFF call atoi
26 0000010B A3[0A000000] mov [B], eax
27
28 00000110 8B0D[35000000] mov ecx, [A]
29 00000116 890D[00000000] mov [max], ecx
30
31 0000011C 3B0D[39000000] cmp ecx, [C]
32 00000122 7F0C jg check_B
33 00000124 8B0D[39000000] mov ecx, [C]
34 0000012A 890D[00000000] mov [max], ecx
35
36 check_B:
37 mov eax, |
37 ***** error: invalid combination of opcode and operands
38 00000130 E867FFFFFF call atoi
39 00000135 A3[00000000] mov [max], eax
40
41 0000013A 8B0D[00000000] mov ecx, [max]
42 00000140 3B0D[0A000000] cmp ecx, [B]
43 00000146 7F0C jg fin
44 00000148 8B0D[0A000000] mov ecx, [B]
45 0000014E 890D[00000000] mov [max], ecx
46
47 fin:
48 00000154 B8[13000000] mov eax, msg2
49 00000159 E8B1FEFFFF call sprint
50 0000015E A1[00000000] mov eax, [max]
51 00000163 E81EFFFFFF call iprintLF
52 00000168 E86EFFFFFF call quit
```

Рис. 4.12: Просмотр ошибки в файле листинга

5 Задания для самостоятельной работы

При выполнении предыдущей лабораторной работы я получила вариант №1, соответственно решаю задания исходя из данных в моём варианте. Для начала, возвращаю операнд к функции в программе и изменяю ее так, чтобы она выводила переменную с наименьшим значением (рис. 5.1).



```
~/work/arch-pc/lab07/lab7-2.asm - Mousepad
File Edit Search View Document Help

%include 'in_out.asm'
SECTION .data
msg1 db 'Введите B: ', 0h
msg2 db 'Наименьшее число: ', 0h
A dd '17'
C dd '45'

SECTION .bss
min resb 10
B resb 10

SECTION .text
GLOBAL _start
_start:

mov eax, msg1
call sprint

mov ecx, B
mov edx, 10
call sread

mov eax, B
call atoi
mov [B], eax

mov ecx, [A]
mov [min], ecx

cmp ecx, [C]
jg check_B
mov ecx, [C]
mov [min], ecx

check_B:
mov eax, min
call atoi
mov [min], eax

mov ecx, [min]
cmp ecx, [B]
jb fin
mov ecx, [B]
mov [min], ecx

fin:
mov eax, msg2
call sprint
mov eax, [min]
call iprintLF
call quit
```

Рис. 5.1: Первая программа самостоятельной работы

Прикладываю код первой программы:

```
%include 'in_out.asm'

SECTION .data
msg1 db 'Введите B: ', 0h
msg2 db 'Наименьшее число: ', 0h
A dd '17'
C dd '45'


SECTION .bss
min resb 10
B resb 10


SECTION .text
GLOBAL _start
_start:

mov eax, msg1
call sprint

mov ecx, B
mov edx, 10
call sread

mov eax, B
call atoi
mov [B], eax

mov ecx, [A]
mov [min], ecx

cmp ecx, [C]
jg check_B
```

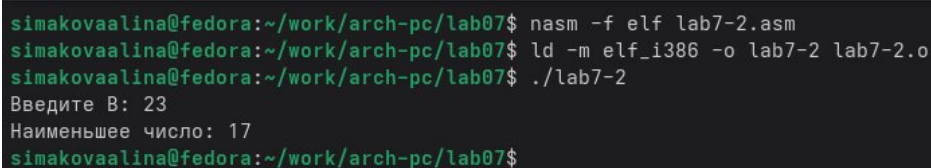
```
mov ecx, [C]
mov [min], ecx
```

```
check_B:
mov eax, min
call atoi
mov [min], eax
```

```
mov ecx, [min]
cmp ecx, [B]
jb fin
mov ecx, [B]
mov [min], ecx
```

```
fin:
mov eax, msg2
call sprint
mov eax, [min]
call iprintLF
call quit
```

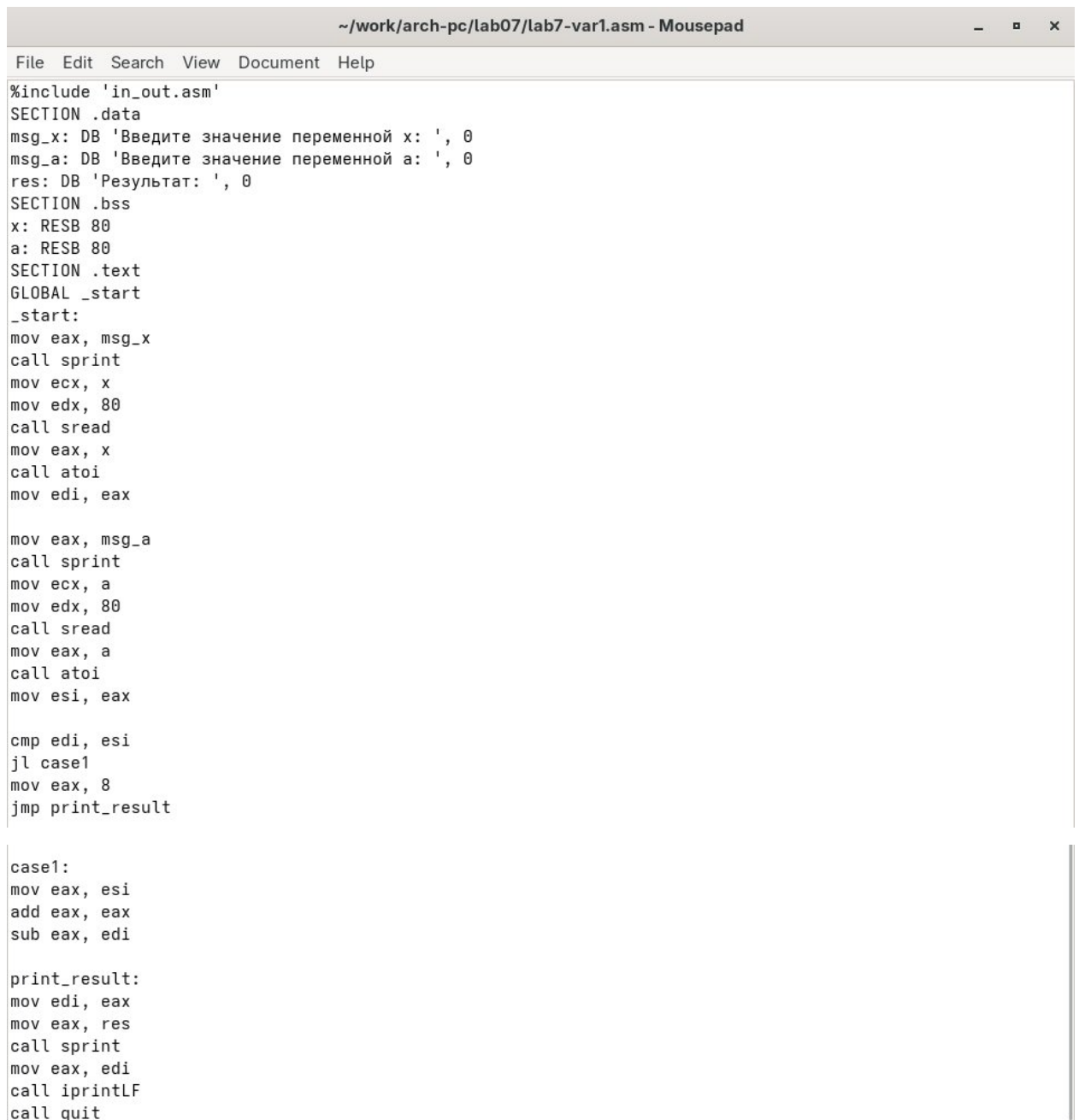
Затем проверяю корректность написания первой программы (рис. 5.2)



```
simakovaalina@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
simakovaalina@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
simakovaalina@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 23
Наименьшее число: 17
simakovaalina@fedora:~/work/arch-pc/lab07$
```

Рис. 5.2: Проверка работы первой программы

Пишу программу, которая будет вычислять значение заданной функции согласно моему варианту для введенных с клавиатуры переменных а и х (рис. 5.3)



```
~/work/arch-pc/lab07/lab7-var1.asm - Mousepad
File Edit Search View Document Help

%include 'in_out.asm'
SECTION .data
msg_x: DB 'Введите значение переменной x: ', 0
msg_a: DB 'Введите значение переменной a: ', 0
res: DB 'Результат: ', 0
SECTION .bss
x: RESB 80
a: RESB 80
SECTION .text
GLOBAL _start
_start:
mov eax, msg_x
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
mov edi, eax

mov eax, msg_a
call sprint
mov ecx, a
mov edx, 80
call sread
mov eax, a
call atoi
mov esi, eax

cmp edi, esi
jl case1
mov eax, 8
jmp print_result

case1:
mov eax, esi
add eax, eax
sub eax, edi

print_result:
mov edi, eax
mov eax, res
call sprint
mov eax, edi
call iprintLF
call quit
```

Рис. 5.3: Вторая программа самостоятельной работы

Прикладываю код второй программы:

```
%include 'in_out.asm'
SECTION .data
msg_x: DB 'Введите значение переменной x: ', 0
msg_a: DB 'Введите значение переменной a: ', 0
res: DB 'Результат: ', 0
SECTION .bss
x: RESB 80
a: RESB 80
```

```

SECTION .text
GLOBAL _start
_start:
mov eax, msg_x
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
mov edi, eax

mov eax, msg_a
call sprint
mov ecx, a
mov edx, 80
call sread
mov eax, a
call atoi
mov esi, eax

cmp edi, esi
jl case1
mov eax, 8
jmp print_result

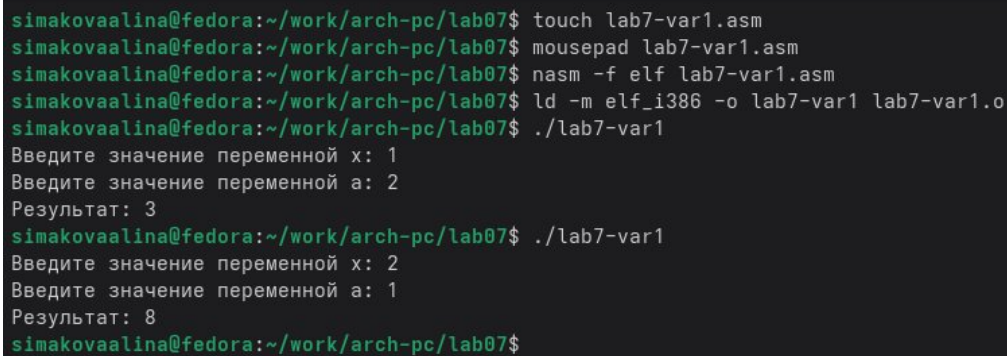
case1:
mov eax, esi
add eax, eax
sub eax, edi

print_result:

```

```
mov edi, eax
mov eax, res
call sprint
mov eax, edi
call iprintLF
call quit
```

Транслирую и компоную файл, запускаю и проверяю работу программы для различных значений а и х (рис. 5.4)



```
simakovaalina@fedora:~/work/arch-pc/lab07$ touch lab7-var1.asm
simakovaalina@fedora:~/work/arch-pc/lab07$ mousepad lab7-var1.asm
simakovaalina@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-var1.asm
simakovaalina@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-var1 lab7-var1.o
simakovaalina@fedora:~/work/arch-pc/lab07$ ./lab7-var1
Введите значение переменной х: 1
Введите значение переменной а: 2
Результат: 3
simakovaalina@fedora:~/work/arch-pc/lab07$ ./lab7-var1
Введите значение переменной х: 2
Введите значение переменной а: 1
Результат: 8
simakovaalina@fedora:~/work/arch-pc/lab07$
```

Рис. 5.4: Проверка работы второй программы

6 Выводы

При выполнении данной лабораторной работы я изучила команды условных и безусловных переходов, приобрела навыки написания программ с использованием переходов и познакомилась с назначением и структурой файлов листинга.

Список литературы

1. <https://esystem.rudn.ru/course/view.php?id=112>
2. https://esystem.rudn.ru/pluginfile.php/2089087/mod_resource/content/0/%D0%9B%D0%B0%D0%B1%D0%BE%D1%80%D0%B0%D1%82%D0%BE%D1%80%D0%BD%D0%B0%D1%8F%20%D1%80%D0%B0%D0%B1%D0%BE%D1%82%D0%B0%20%E2%84%967.%20%D0%9A%D0%BE%D0%BC%D0%B0%D0%BD%D0%B4%D1%8B%20%D0%B1%D0%B5%D0%B7%D1%83%D1%81%D0%BB%D0%BE%D0%B2%D0%BD%D0%BE%D0%B3%D0%BE%20%D0%B8%20%D1%83%D1%81%D0%BB%D0%BE%D0%B2%D0%BD%D0%BE%D0%B3%D0%BE%20%D0%BF%D0%B5%D1%80%D0%B5%D1%85%D0%BE%D0%B4%D0%BE%D0%B2%20%D0%B2%20Nasm.%20%D0%9F%D1%80%D0%BE%D0%B3%D1%80%D0%B0%D0%BC%D0%BC%D0%B8%D1%80%D0%BE%D0%B2%D0%B0%D0%BD%D0%B8%D0%B5%20%D0%B2%D0%B5%D1%82%D0%B2%D0%BB%D0%B5%D0%BD%D0%B8%D0%B9..pdf
3. <https://esystem.rudn.ru/mod/page/view.php?id=1030492>
4. <https://esystem.rudn.ru/mod/resource/view.php?id=1030496>